

1. Write a short note on Goal based agent with a diagram

2. Define pruning? Explain alpha and beta pruning algorithm

3. Differentiate between first-order logic and propositional logic.

Feature	Propositional Logic	First-Order Logic
Basic Unit	Propositions	Predicates, constants, variables
Expressiveness	Limited to true/false statements	Expressive, can represent relationships and properties
Quantifiers	None	Universal ($\forall$) and Existential ($\exists$)
Syntax	Combines propositions using logical connectives	Uses predicates and quantifiers
Semantics	Truth tables	Interpretation over a domain
Use Cases	Simple problems (e.g., circuit design, rule-based systems)	Complex problems (e.g., AI reasoning, ontology modeling)
Example	$P \rightarrow Q$	$\forall x \exists y (Likes(x, y))$

#SEE NOTEBOOK

4. Define Belief Network. Describe the steps of constructing a belief network with an example. What types of inferences can be drawn from that?

Definition

A Belief Network (Bayesian Network) is a Directed Acyclic Graph (DAG) that represents variables as nodes and their conditional dependencies as directed edges. It uses Conditional Probability Tables (CPTs) to quantify these relationships.

Steps to Construct the Belief Network (from your example)

1. Identify the Variables:

In this example, the variables are Humid (H), Cloud (C), Rain (R), Wet Grass (W), and School Off (O).

2. Determine Causal Relationships:

- Humidity and Clouds are independent factors that influence Rain.
- Rain directly influences whether the Grass is Wet and whether School is Off.

3. Draw the Network Structure (Nodes and Edges):

- **Root Nodes:** Create nodes for Humid and Cloud (no incoming arrows).
- **Intermediate Node:** Create a Rain node with arrows pointing from Humid and Cloud to Rain.
- **Leaf Nodes:** Create Wet Grass and School Off nodes with arrows pointing from Rain to each.

4. Quantify with Conditional Probability Tables (CPTs):

Assign the probability values for each scenario:

- **Independent Probabilities:** $P(H) = 0.2$ and $P(C) = 0.6$.
- **Joint Probabilities (Rain):** Fill the CPT for Rain based on all combinations of H and C (e.g., $P(R | H, C) = 0.9$).
- **Dependent Probabilities:** Define probabilities for the effects (e.g., $P(W|R) = 0.8$, meaning if it rains, there is an 80% chance the grass is wet).

Types of Inferences from this Example

1. **Causal Inference (Top-Down):**
 - **Scenario:** It is Humid and Cloudy. What is the probability that it will Rain?
 - **Direction:** From parents (H, C) to child (R).
2. **Diagnostic Inference (Bottom-Up):**
 - **Scenario:** The grass is wet. What is the probability that it actually Rained?
 - **Direction:** From effect (W) back to cause (R).
3. **Intercausal Inference (Explaining Away):**
 - **Scenario:** It is Raining. If we discover it is very Cloudy, the probability that high Humidity was the cause "decreases" because the clouds explain the rain.
 - **Direction:** Between two causes (H and C) of the same effect (R).
4. **Mixed Inference:**
 - **Scenario:** Knowing the grass is wet (Diagnostic) and it is not cloudy (Causal), what is the probability of it being Humid?

#NOTEBOOK

5. Discuss A* Algorithm with an example.

1. Definition

A* is a best-first search algorithm that finds the shortest path from a given starting node to a target goal node. It is widely used in maps, GPS systems, and video game character movement.

2. The Core Formula

The algorithm decides which path to take by calculating a "cost" for each node using the following equation:

$$f(n) = g(n) + h(n)$$

- **$g(n)$** : The actual cost of the path from the **start node** to the current node n .
- **$h(n)$** : The **heuristic value**, which is an estimated cost from node n to the **goal**.
- **$f(n)$** : The total estimated cost of the lowest-cost solution through node n .

3. How A* Works (Step-by-Step)

1. **Initialize**: Place the starting node in an **Open List** (nodes to be explored).
2. **Select**: Pick the node with the lowest $f(n)$ value from the Open List.
3. **Expand**: Move this node to the **Closed List** (already explored). Check its neighbors.
4. **Update**: For each neighbor, calculate its g , h , and f values. If a neighbor is already in the Open List with a higher cost, update it with the lower cost.
5. **Repeat**: Continue this process until the goal node is reached or the Open List is empty (meaning no path exists).

4. Key Properties (Important for Exams)

- **Completeness**: A* is **complete**, meaning it will always find a path if one exists.
- **Optimality**: A* is **optimal** (finds the shortest path) if the heuristic $h(n)$ is **admissible**.
 - *Admissible* means the heuristic never overestimates the actual cost to reach the goal.
- **Efficiency**: It is more efficient than Dijkstra's Algorithm because it uses the heuristic to ignore paths that head in the wrong direction.

5. Heuristic Functions

The choice of $h(n)$ depends on the problem:

- **Manhattan Distance**: Used when movement is restricted to 4 directions (up, down, left, right).
 - **Euclidean Distance**: The straight-line "as the crow flies" distance, used when movement can be in any angle.
-

6.Explain the concept of hill climbing algorithm & simulated annealing algorithm.

Local Search Algorithms: AI Notes

1. Hill Climbing Algorithm

Definition: A greedy local search algorithm that continuously moves in the direction of increasing value (upland) to find the peak of a mountain or the best solution. It terminates when it reaches a state where no neighbor has a higher value.

Key Concepts

- **Initial State:** Starts with an arbitrary solution.
- **Neighbor Evaluation:** Checks the immediate neighbors of the current state.
- **Greedy Choice:** Moves only if the neighbor is better than the current state.
- **Termination:** Stops at a "Local Optimum" where no further improvement is possible.

The Search Landscape (Common Obstacles)

- **Local Maxima:** A peak that is higher than its neighbors but lower than the global maximum.
- **Plateau:** A flat area where all neighbors have the same value, making it difficult to find a direction.
- **Ridges:** A sequence of local maxima where the surface slopes in all directions except the top of the ridge.

Variants

1. **Simple Hill Climbing:** Moves to the *first* neighbor that is better than the current state.
2. **Steepest-Ascent:** Evaluates *all* neighbors and chooses the one with the highest improvement.
3. **Stochastic Hill Climbing:** Chooses at random from among the better neighbors.

2. Simulated Annealing Algorithm

2. Simulated Annealing Algorithm

Definition: A probabilistic optimization algorithm inspired by the physical process of **annealing in metallurgy** (heating and slow cooling of metals). It avoids getting stuck in local optima by occasionally accepting "worse" moves.

The Annealing Concept

- **High Temperature (T):** At the start, the algorithm "explores" the landscape by accepting worse moves with high probability.
- **Cooling Schedule:** As T gradually decreases, the algorithm becomes more "greedy," eventually settling into a global optimum.

Mathematical Mechanism

1. **Energy Difference (ΔE):**

$$\Delta E = Cost(S_{neighbor}) - Cost(S_{current})$$

2. **Acceptance Probability (P):** If a move is worse ($\Delta E > 0$), it is accepted only if a random number between 0 and 1 is less than:

$$P = e^{-\frac{\Delta E}{T}}$$

3. **Cooling Function:** The temperature is reduced in each iteration using a constant α :

$$T_{new} = \alpha \times T_{old} \quad (\text{where } 0 < \alpha < 1)$$

Comparison Table: Hill Climbing vs. Simulated Annealing

Feature	Hill Climbing	Simulated Annealing
Strategy	Purely Greedy	Probabilistic / Metaheuristic

Move Choice	Only moves "upward"	Can move "downward" to explore
Local Maxima	Often gets stuck	Designed to escape local maxima
Parameters	None (Simple)	Requires T and Cooling Rate (α)
Outcome	Local Optimum	Likely to find Global Optimum
Analogy	Climbing in the fog	Metal cooling to a stable state

7. Write a short note on Statistical learning.

Definition

Statistical Learning is a branch of machine learning that focuses on understanding and modeling the relationship between input variables (features) and output variables (target) using statistical methods.

Key Idea

Statistical learning aims to learn a function of the form:

$$Y = f(X) + \varepsilon$$

Where:

- X = Input variables
- Y = Output variable

- ε = Random error

Goals:

- Prediction (accurate estimation of output)
 - Inference (understanding relationship between variables)
-

Types of Statistical Learning

1. Supervised Learning
 - Uses labeled data
 - Examples: Regression, Classification
 2. Unsupervised Learning
 - Uses unlabeled data
 - Examples: Clustering, Dimensionality Reduction
-

Common Techniques

- Linear Regression
 - Logistic Regression
 - Decision Trees
 - k-Nearest Neighbors (KNN)
-

Applications

- Spam detection
 - Medical diagnosis
 - Stock market analysis
 - Recommendation systems
-

Advantages

- Strong mathematical foundation
- Helps in interpreting data
- Useful for prediction and analysis

Limitations

- Assumes certain data distributions
- Sensitive to noise and outliers

Conclusion

Statistical learning combines statistics and machine learning to build models that can predict outcomes and extract meaningful insights from data.